

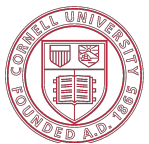
EqMap: FPGA LUT Remapping with E-Graphs

Matthew Hofmann, Berk Gokmen, Zhiru Zhang

Cornell University

ICCAD 2025

Munich, 27 October 2025

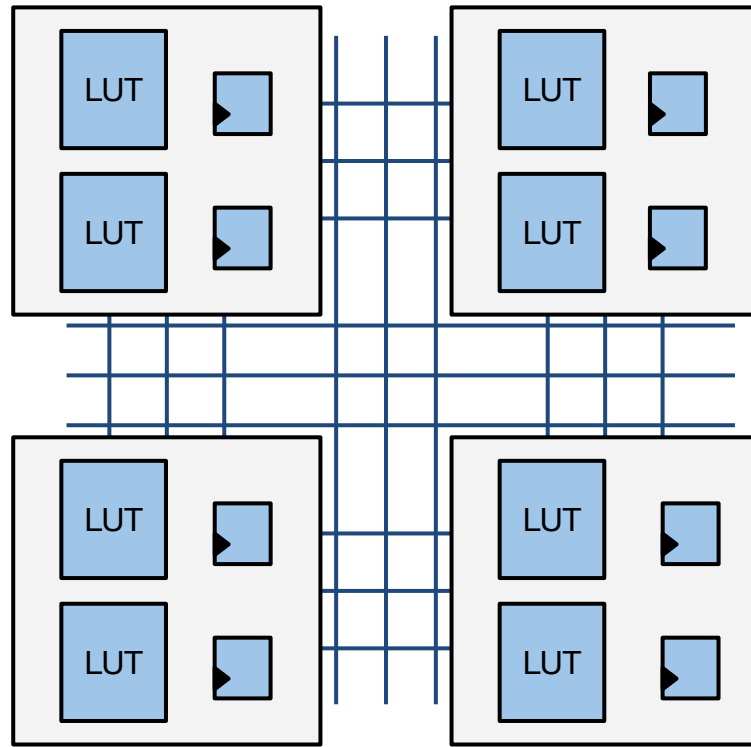


Cornell University

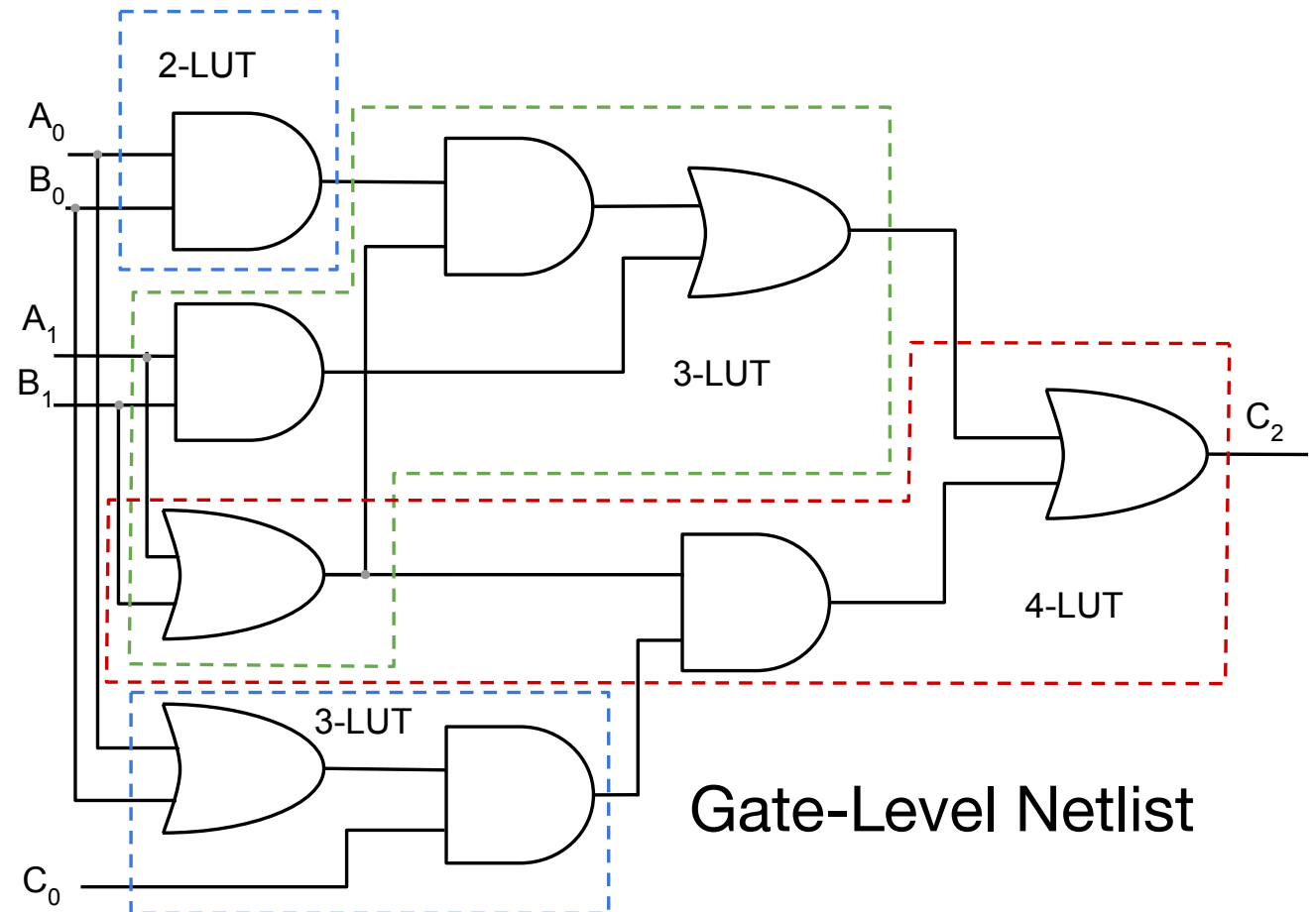


FPGA Technology Mapping

- ▶ FPGA compilation maps technology independent logic networks to an efficient assembly of lookup tables (LUTs)

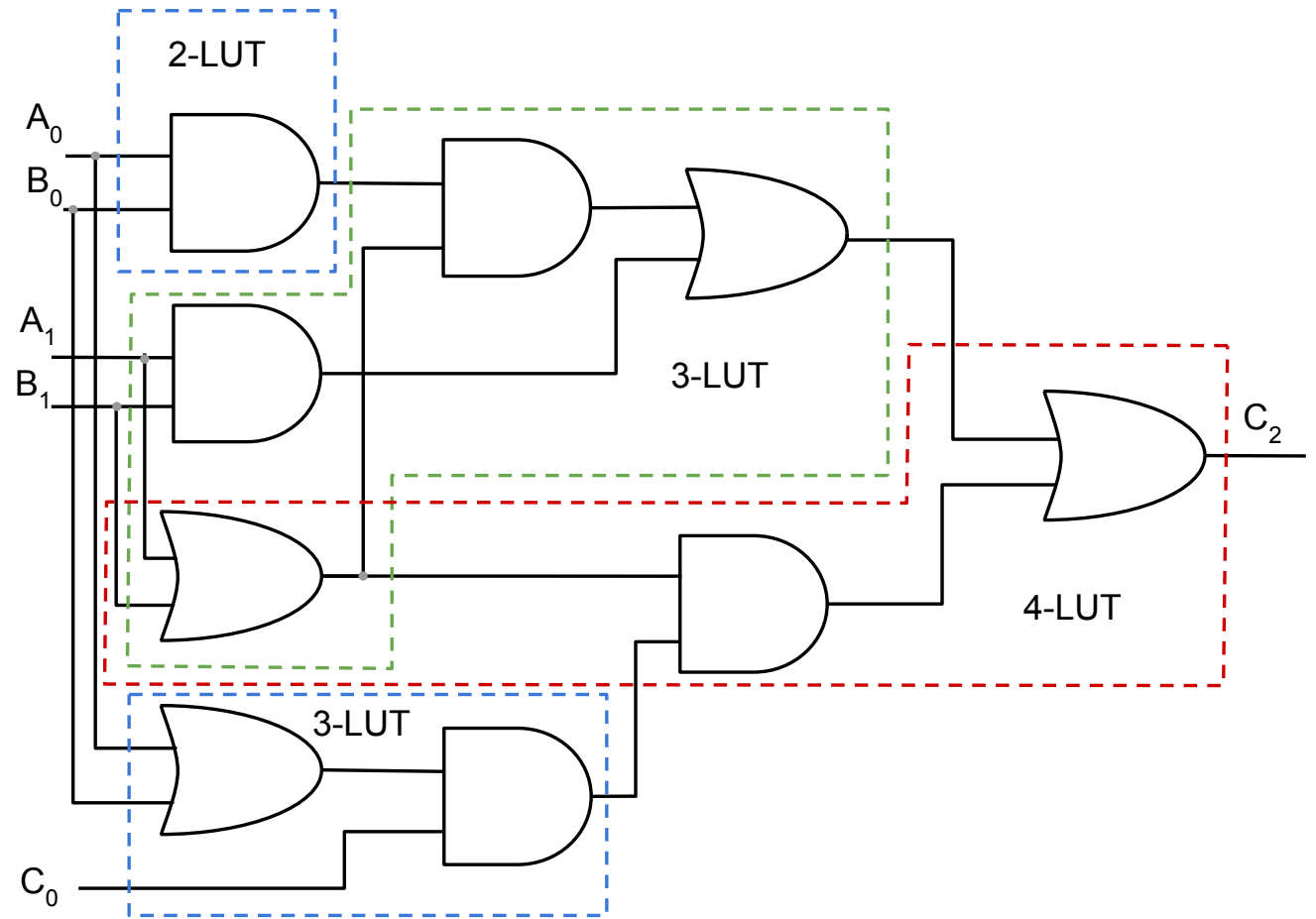


Basic FPGA Fabric



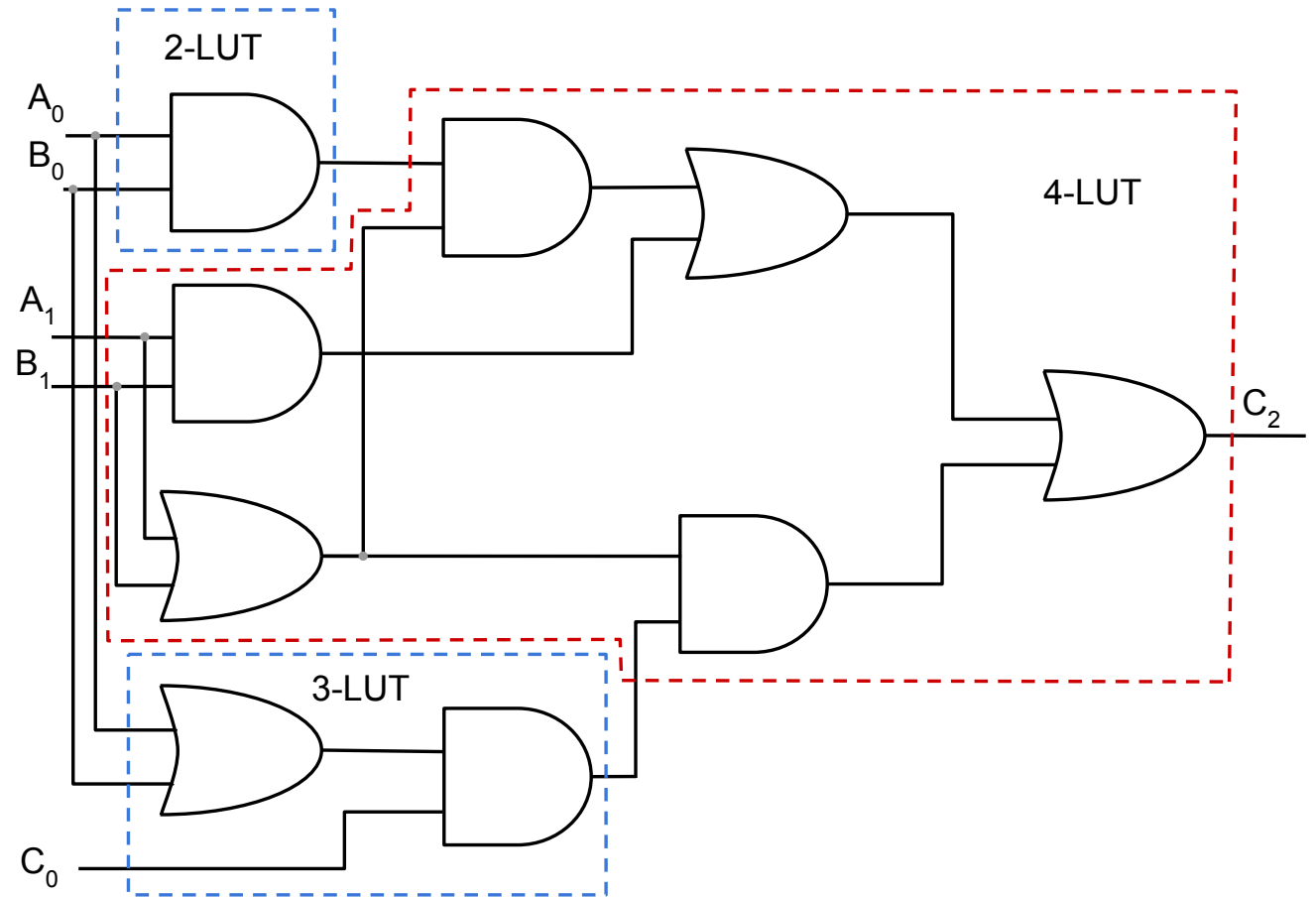
Difficulty of FPGA Technology Mapping

- ▶ Finding the optimal circuit mapping w.r.t. area is NP-hard
- ▶ **Functional** mappers find the best decomposition of the circuit into Boolean functions
- ▶ **Structural** mappers find the best covering of the input graph with LUTs
- ▶ Choosing the best cover
 - Non-monotone clustering



Difficulty of FPGA Technology Mapping

- ▶ Finding the optimal circuit mapping is NP-hard
- ▶ **Functional** mappers find the best decomposition of the circuit into Boolean functions
- ▶ **Structural** mappers find the best covering of the input graph with LUTs
- ▶ Choosing the best cover
 - Non-monotone clustering
- ▶ **Structural bias**
 - High degree of symmetry



Less Heuristic Technology Mapping

- ▶ **Challenge:** Heuristic algorithms do not search a wide enough space
 - Our results support this generalization
- ▶ **Objective:** Can we use existing tech mapping heuristics as a starting point?
 - Make heuristics more resilient to structural bias
- ▶ **Solution:** E-Graphs assist logic synthesis with non-destructive rewriting.
 - Equality graphs use union-find operations to reason about equivalence relations.
 - LUT rewriting w/ e-graphs can discover simpler technology mappings
 - The process is called **equality saturation**: $\text{EqSat} + \text{Tech Mapping} = \text{EqMap}$

E-Graphs as a Data Structure

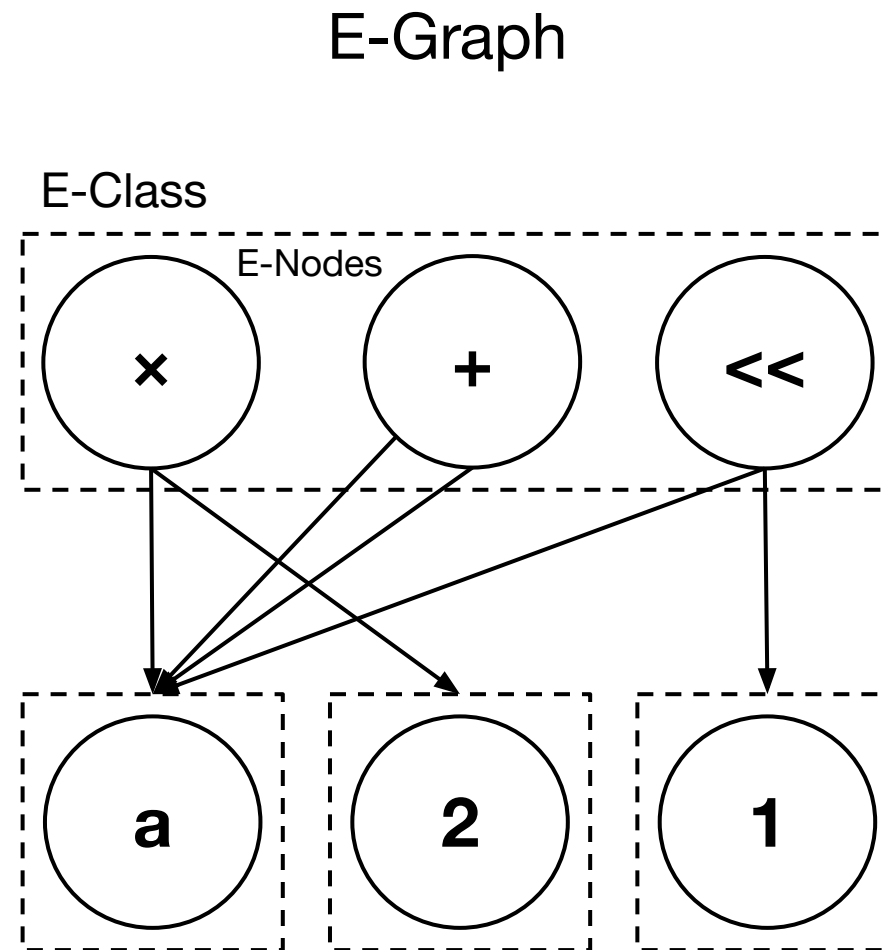
- ▶ Two types: e-node and e-class
- ▶ Edges: e-node to e-class
- ▶ Allows compact representation of equivalent terms

Original Program

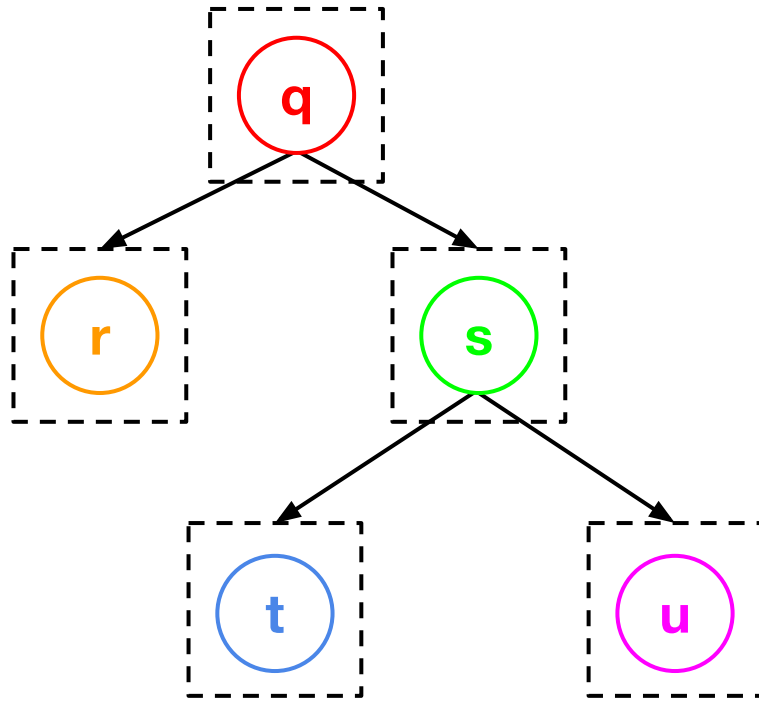
```
int times_two(int a) {  
    return a + a;  
}
```

Properties (Rewrites)

```
a + a = a * 2  
a * 2 = a << 1
```

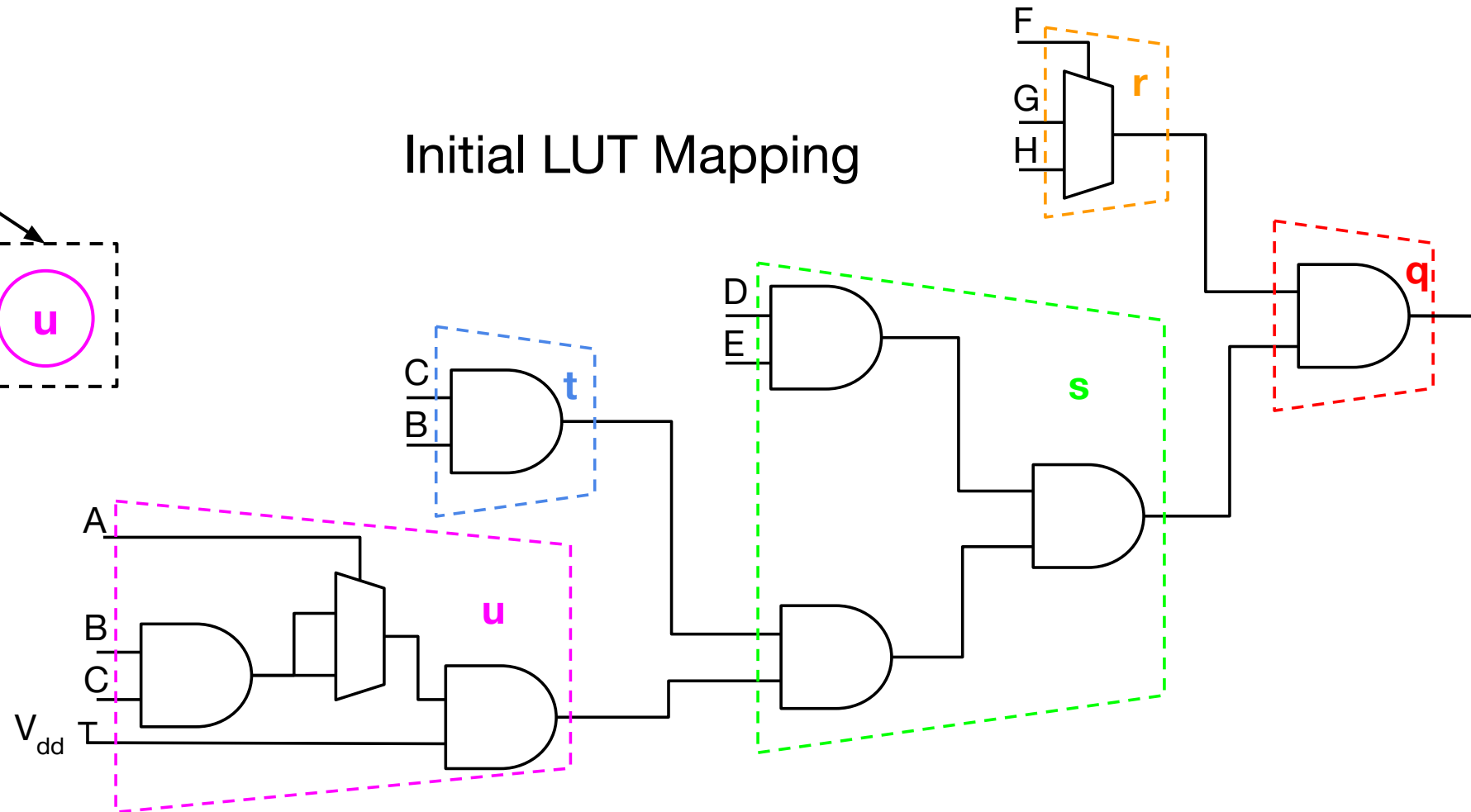


Building The Initial E-Graph

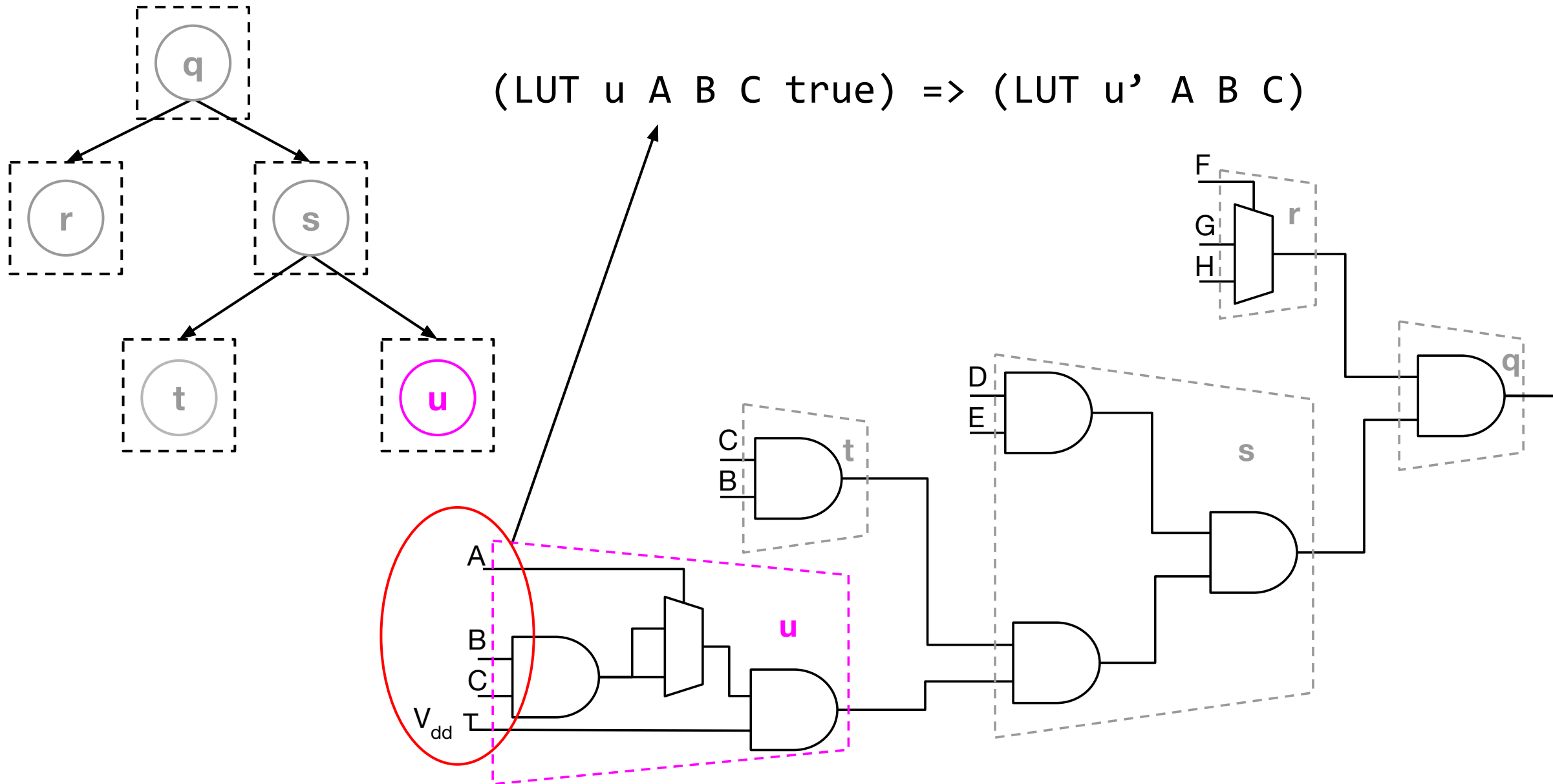


Initial E-Graph

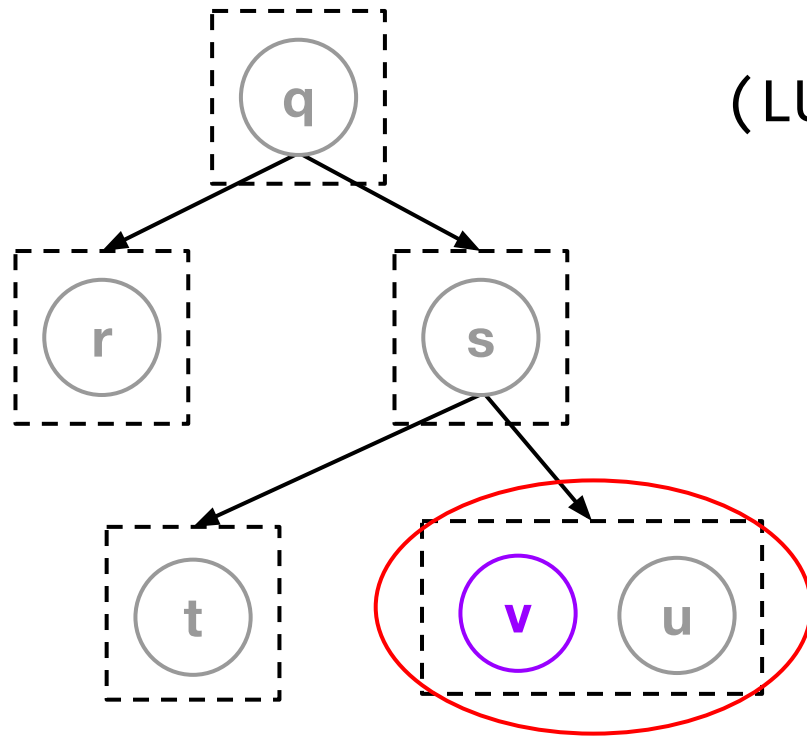
Initial LUT Mapping



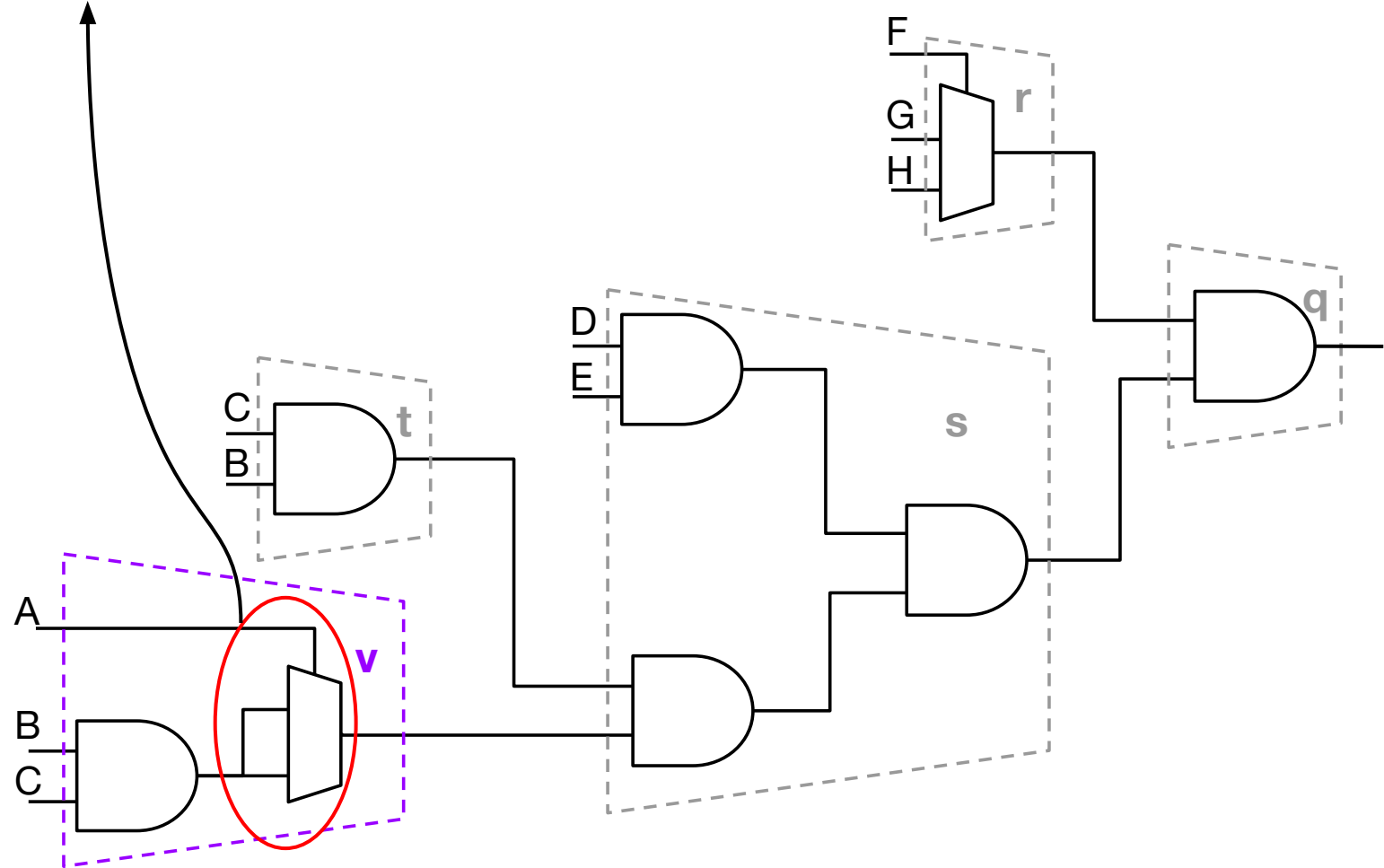
LUT Rewriting System: Constant Inputs



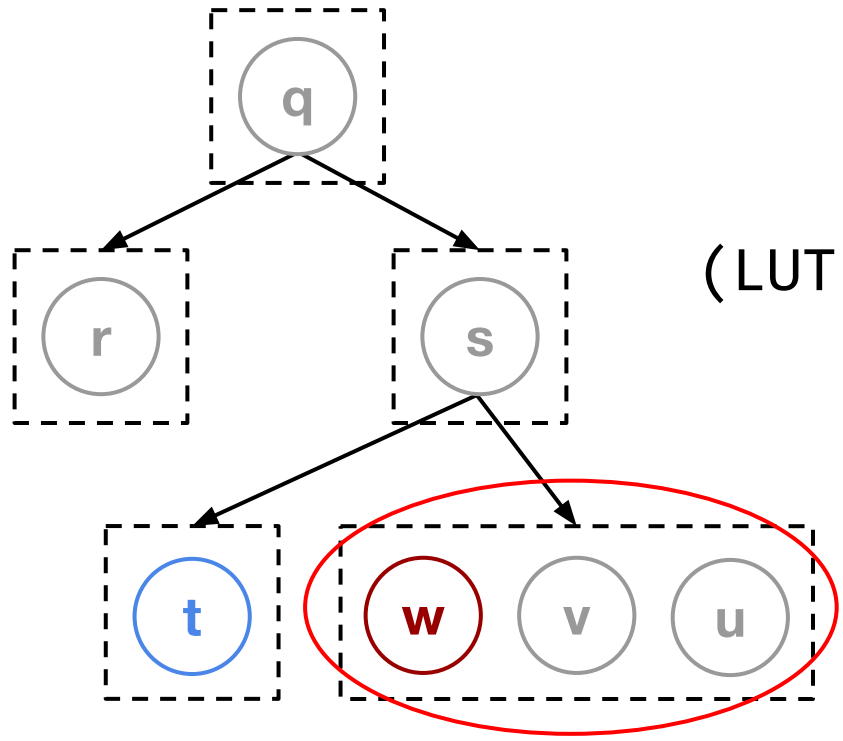
LUT Rewriting System: Invariance



$(\text{LUT } v \ A \ B \ C) \Rightarrow (\text{LUT } v' \ B \ C)$
if $v(1, b, c) := v(0, b, c)$



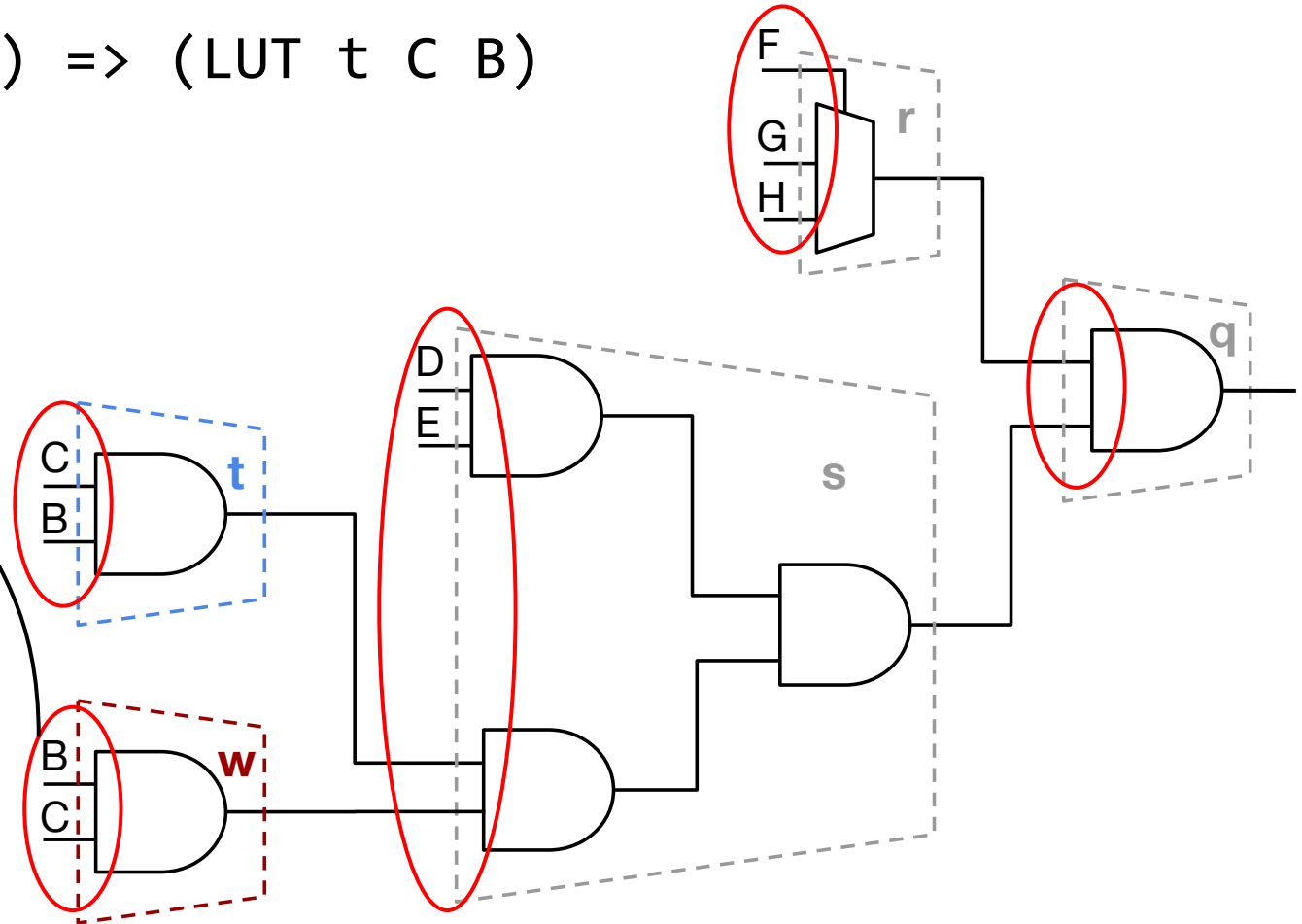
LUT Rewriting System: Symmetry



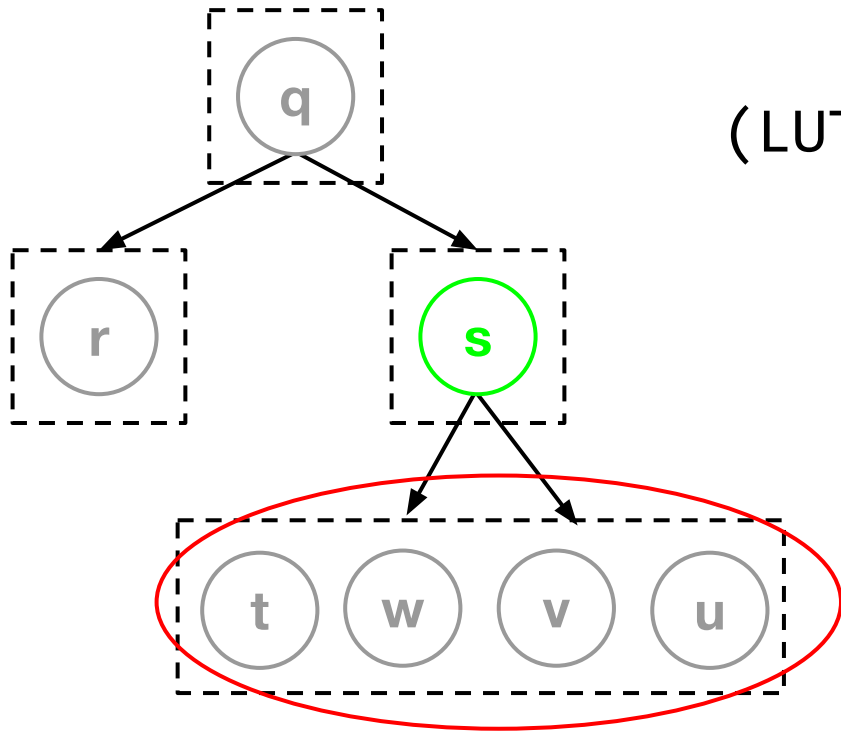
$(\text{LUT } w \ B \ C) \Rightarrow (\text{LUT } t \ C \ B)$

We can always re-assign pins.

But rewriting **w** will create **t**.

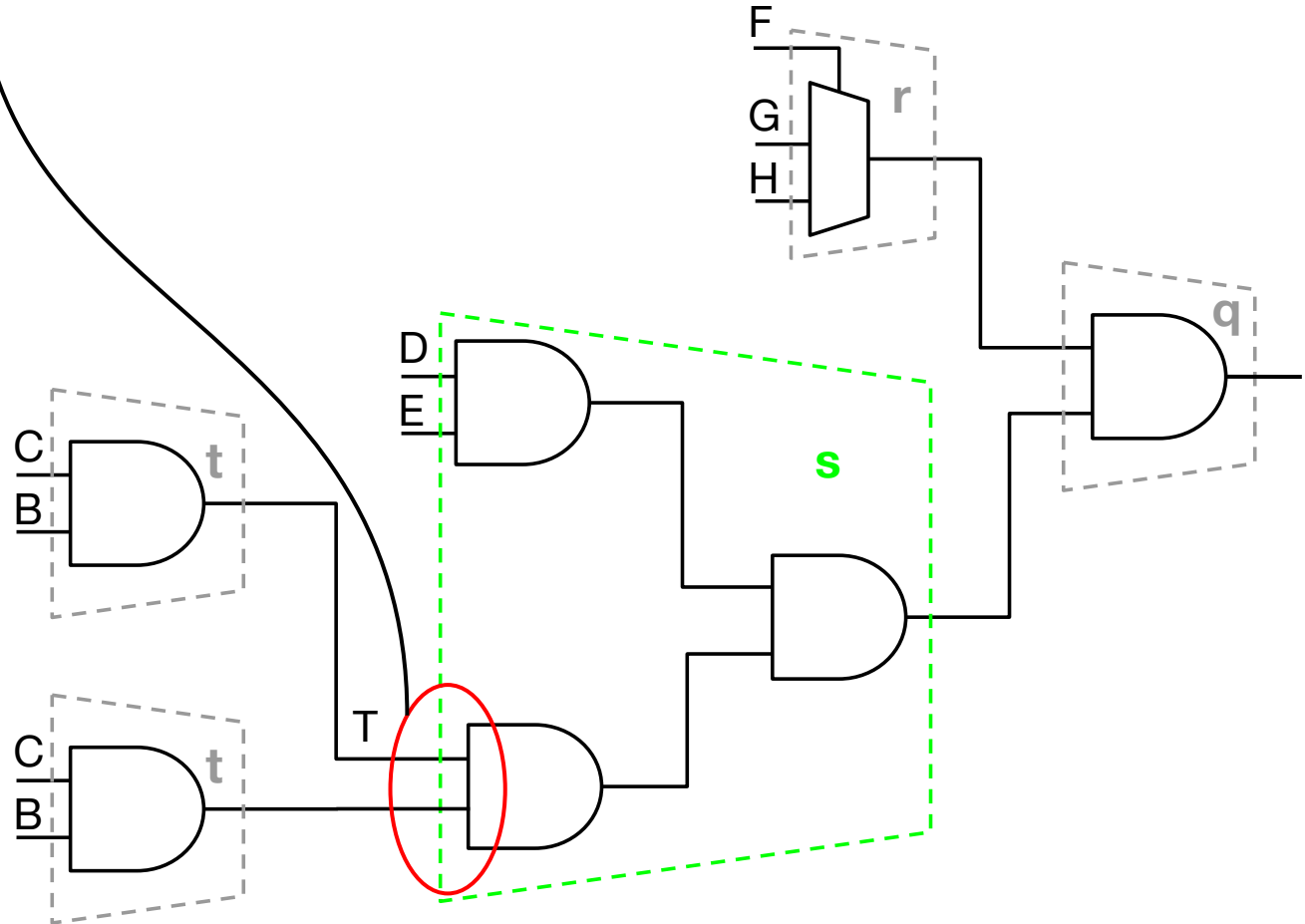


LUT Rewriting System: Restricted Domain

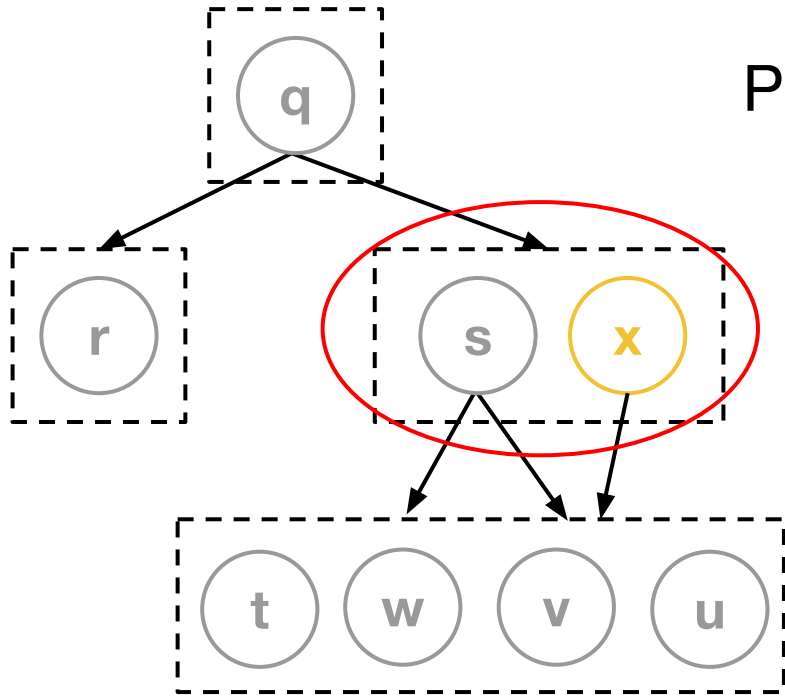


E-Graph proved equivalent terms, causing e-classes to merge.

$$(LUT\ s\ D\ E\ T\ T) \Rightarrow (LUT\ s'\ D\ E\ T)$$

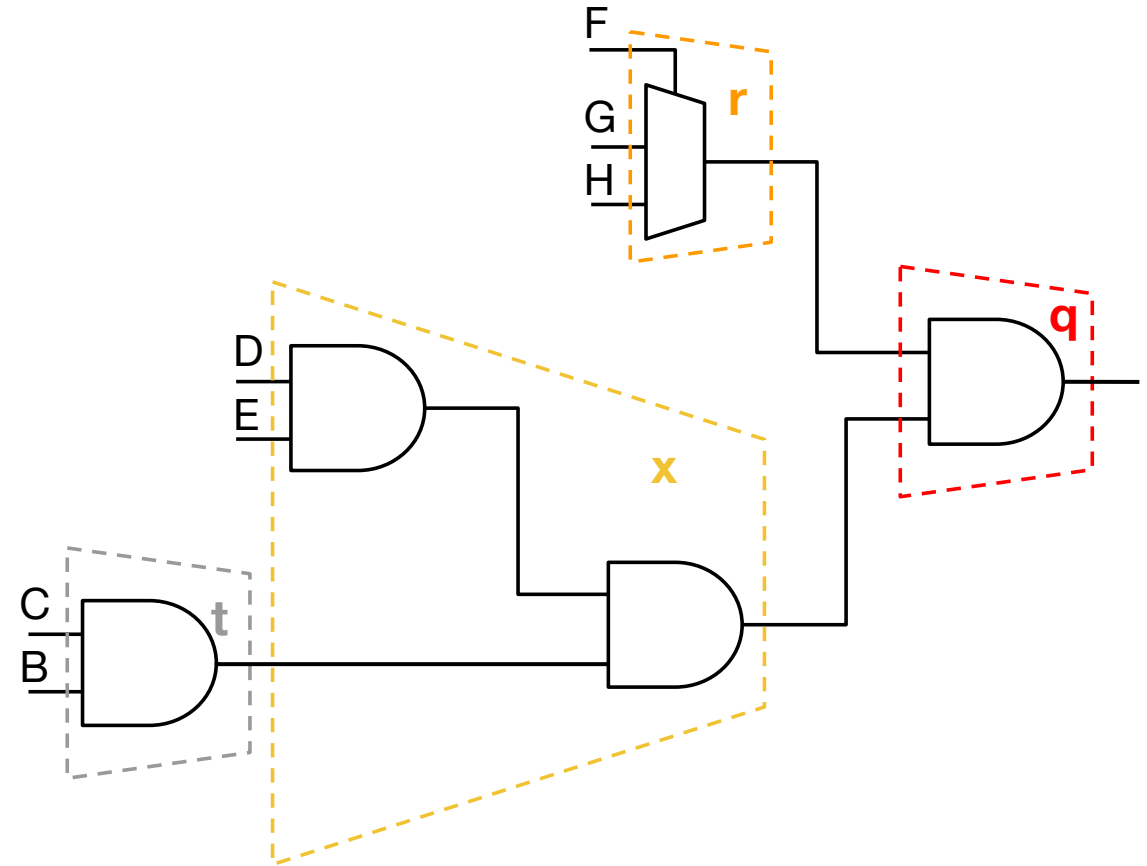


LUT Rewriting System: Cut Composition

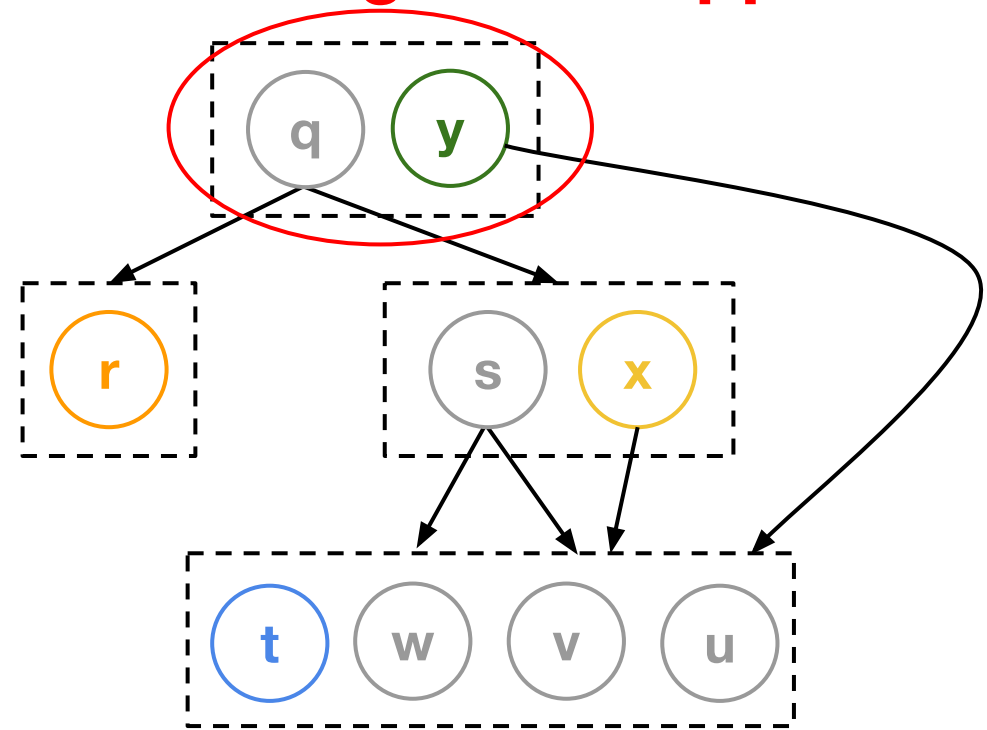


Proved this cut is actually 3-feasible.

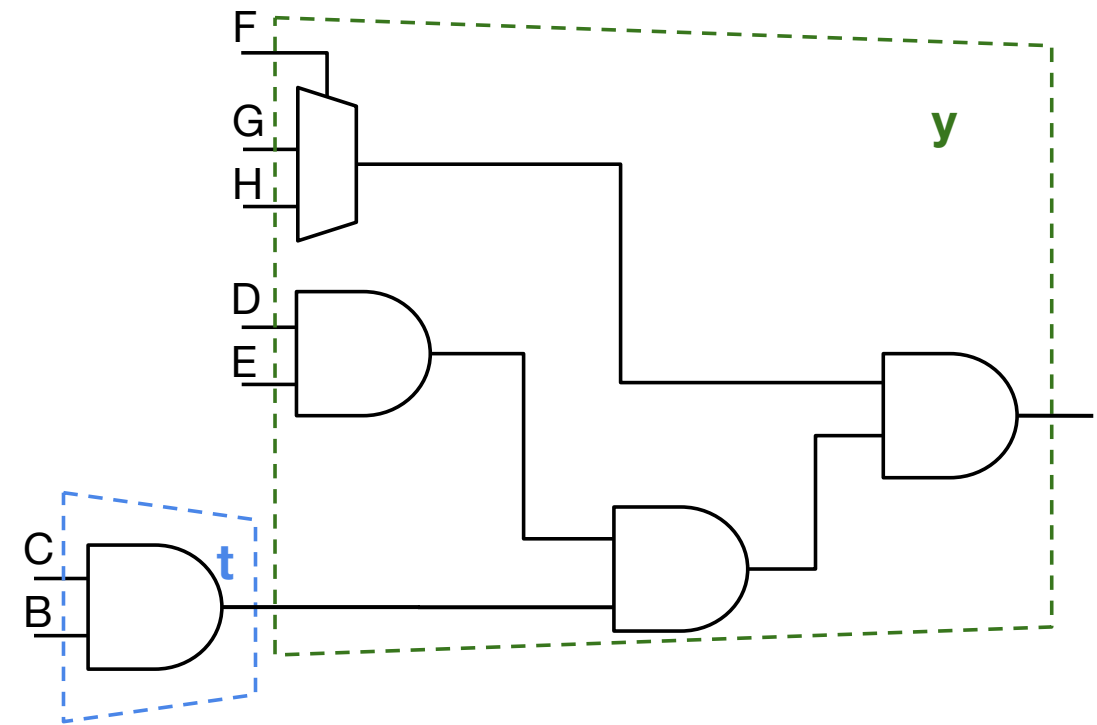
Consequently, $\{q, r, x\}$ is now 6-feasible.



Extracting a Remapped Solution



Extraction	LUT Count
{q, r, x, t}	4
{y, t}	2



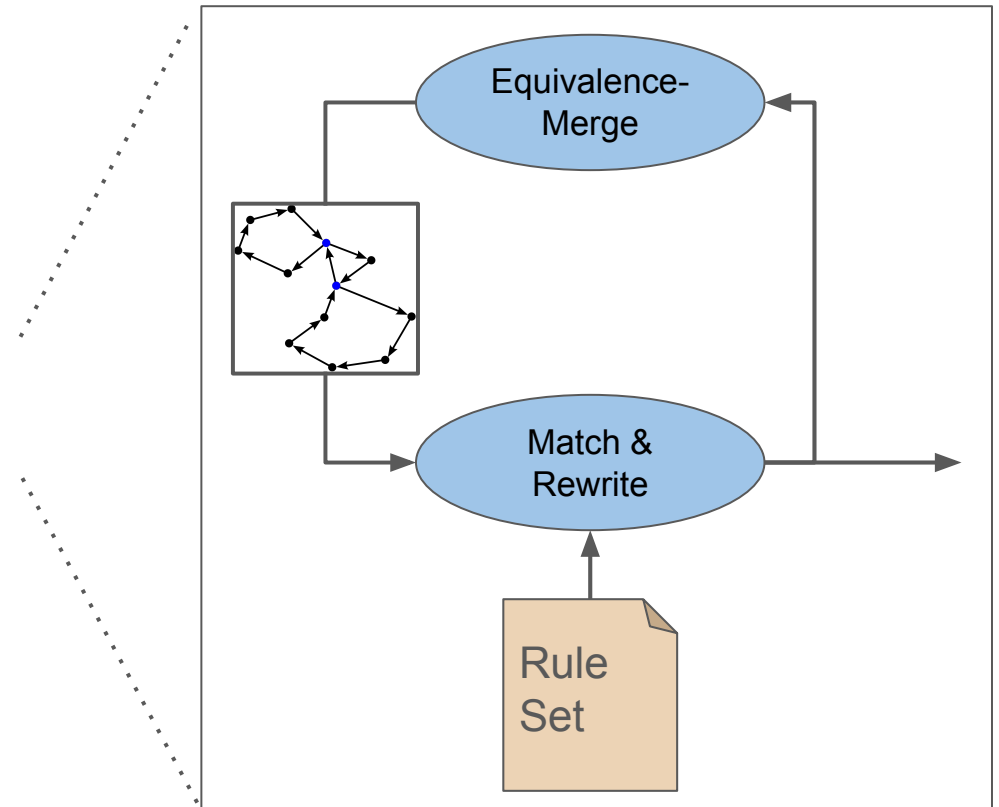
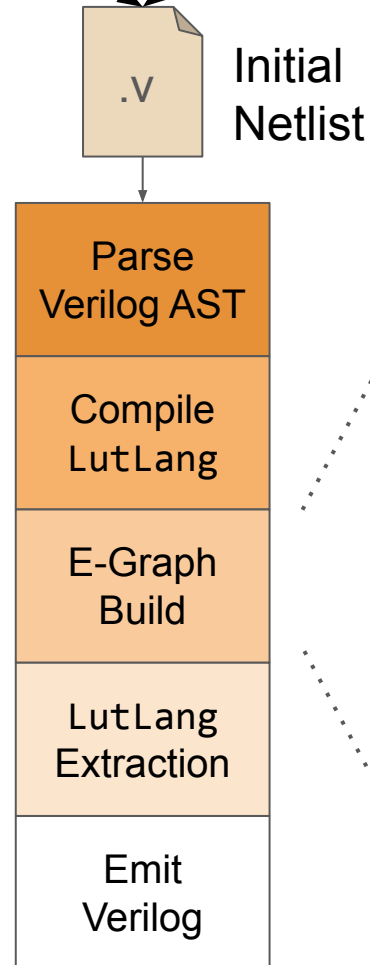
Rewriting System as a Whole

- ▶ Rewrites are applied in every order
- ▶ Macros generate the rewrite patterns for different sized k-LUTs
- ▶ Rewrites operate on truth tables (not gates)
- ▶ In total, 71 rewrite rules

Rule Type	Purpose	Count
LUT Symmetry	Make other rewrite rules independent of input positions	15
Domain Restriction	Simplify a LUT with redundant inputs	5
Cut Composition	Reduce the number of LUTs in a cut when it is k-feasible	36
Inverter Insertion	Allow a node or its complement to be used	1
Register Retiming	Spread timing slack to more critical paths	6
Misc	Constant truth tables, simple gates, etc..	8

E-Graph Remapping Flow

- ▶ Input is FPGA netlist
 - From Vivado
 - From Yosys
- ▶ Output is remapped netlist
- ▶ The same reasoning techniques could be used for ASIC design
 - Boolean/LUT rewriting
 - Algebraic rewriting
- ▶ Rewrites and re-merges are batched into “iterations”



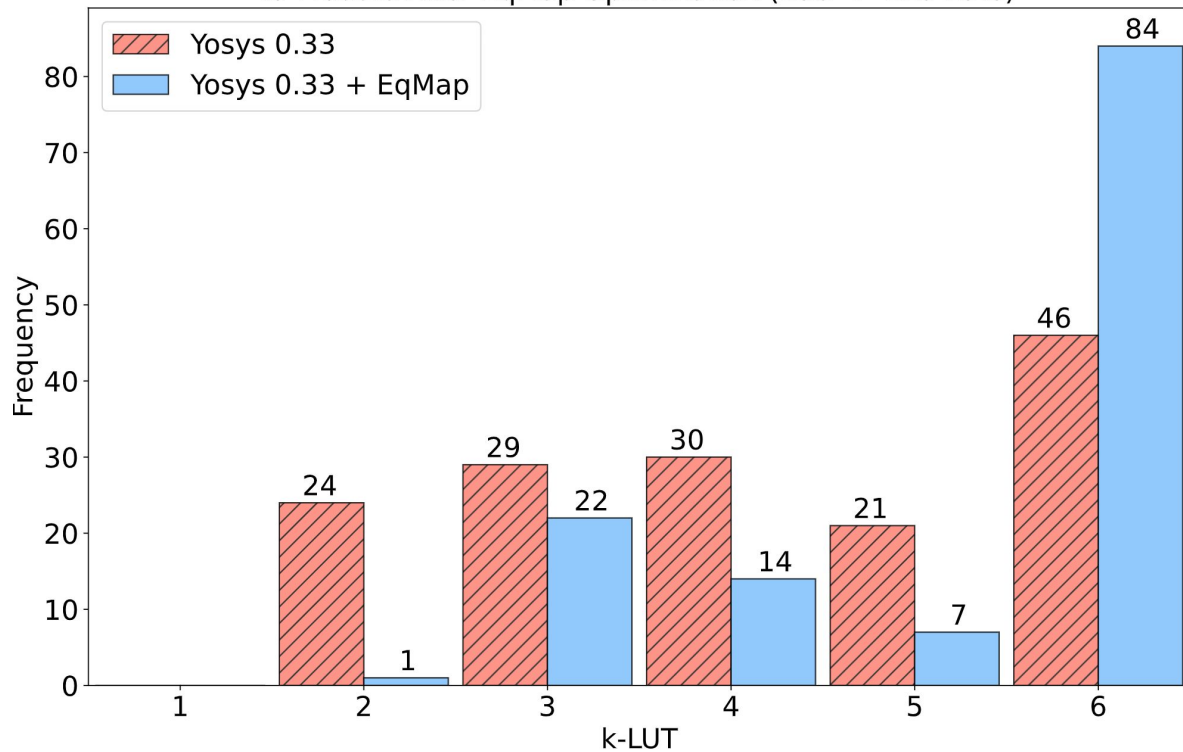
Results

- ▶ Benchmarks from ISCAS'85, LGSynth'91, and EPFL
 - 95 total
 - 42 improved
 - No regressions
- ▶ Several examples leapfrog Vivado
- ▶ High Yosys LUT count → Low EqMap LUT Count
- ▶ E-Graph build: < 5s
- ▶ Extraction: < 300ms

Module	LUT Count		
	EqMap	Vivado 2024	Yosys 0.33
mem_ctrl	10566 (-9.35%)	11697	11101
div	5050 (-64.62%)	14337	5319
square	3497 (-14.46%)	4088	4009
arbiter	2655 (-3.21%)	2743	2655
sin	1494 (-7.15%)	1609	1531
voter	1440 (-10.78%)	1614	1461
max	798 (-0.25%)	798	842
i10	559 (-10.99%)	628	580
k2	520 (-0.38%)	522	537
c6288	513 (-24.67%)	681	519
c7552	328 (-8.38%)	358	333
c5315	262 (-6.76%)	281	263
adder	259 (-20.55%)	326	276
dalu	216 (-11.11%)	243	237
vda	204 (-2.86%)	210	270
frg2	199 (-9.13%)	219	202
rot	176 (-6.88%)	189	196
apex6	166 (-9.29%)	183	168
x3	157 (-5.99%)	167	161
alu4	132 (-2.94%)	136	204
i9	128 (-3.76%)	133	150
cavlc	92 (-13.21%)	106	134
c1908	89 (-3.26%)	92	94

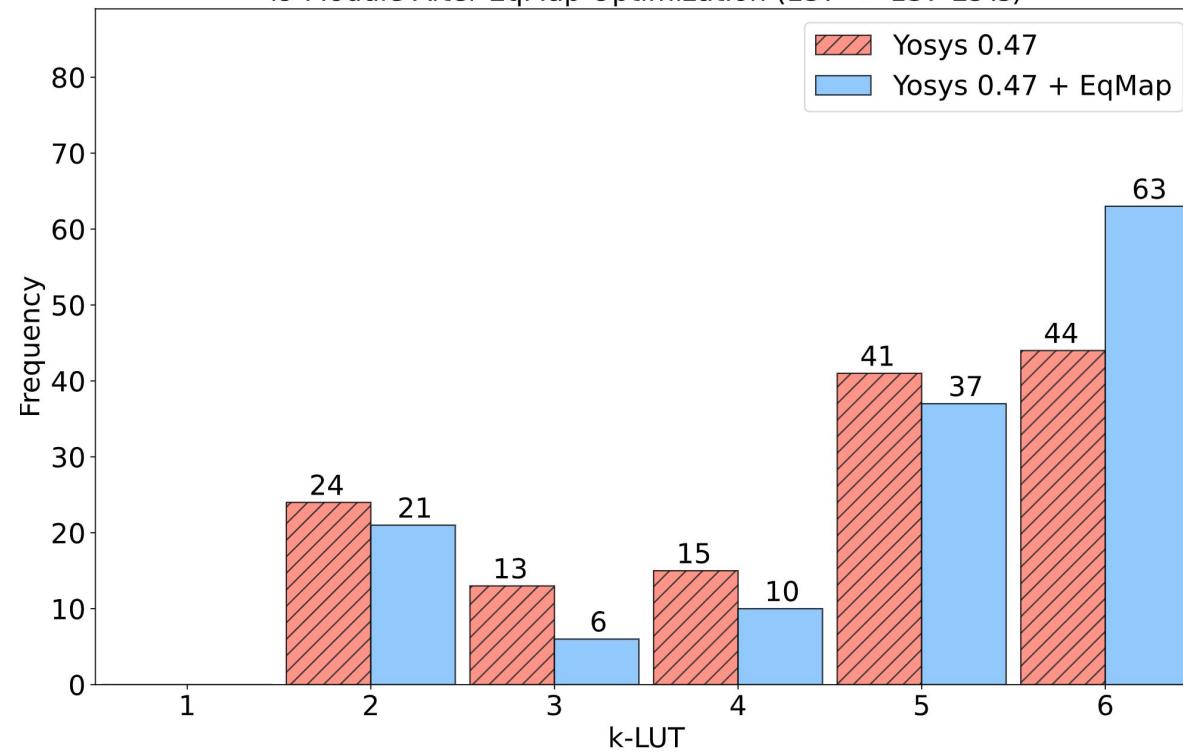
Results (cont.)

i9 Module After EqMap Optimization (150 -> 128 LUTs)



EqMap increases 6-LUT utilization

i9 Module After EqMap Optimization (137 -> 137 LUTs)



EqMap does not remove all structural bias

Future Work

- ▶ Better support for sequential designs
- ▶ Optimizing for timing
- ▶ Code refactors, modularize compiler components
- ▶ **E-Graph Extraction**
 - Reaching “equality saturation” means the e-graph contains our optimal solution
 - But how do we obtain/choose it? NP-complete problem
 - Other research explores extraction heuristics

EqMap: FPGA LUT Remapping using E-Graphs

- ▶ Post-Technology Mapping Optimization via E-Graphs
 - “Correct by construction” software implementation
 - Verifiable w/ automated proof generation
- ▶ 12% fewer LUTs than commercial tools on improved benchmarks
 - 6% fewer over all benchmarks
 - No regressions, no increase in circuit depth
 - Modest increases in runtime (seconds)
- ▶ Open source
 - Packaged as a Verilog-to-Verilog tool
 - Written in Rust



[cornell-zhang / eqmap](https://github.com/cornell-zhang/eqmap)



In General, Exact Synthesis is Intractable

- ▶ SAT-based synthesis precisely captures the circuit simplification problem
- ▶ Other Boolean matching and Boolean resubstitution tricks scale a bit better
- ▶ Generally cannot be scaled past the tens of gates

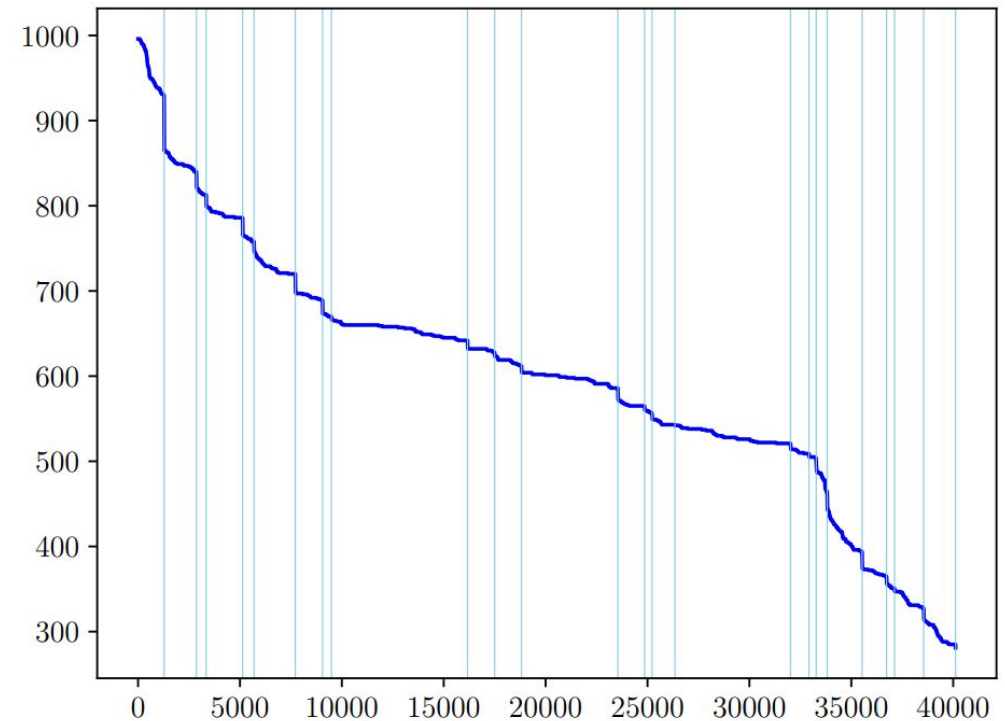
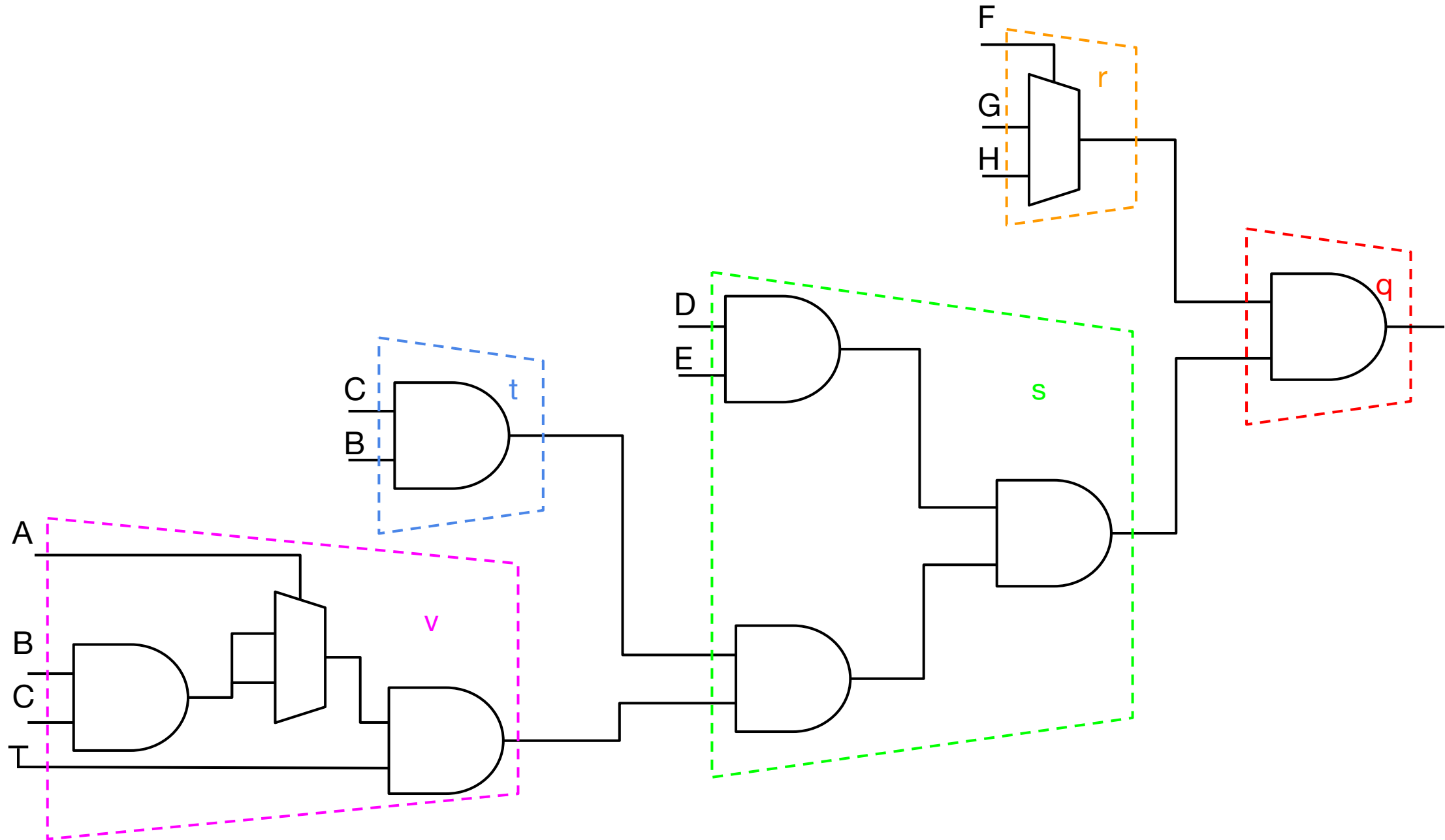


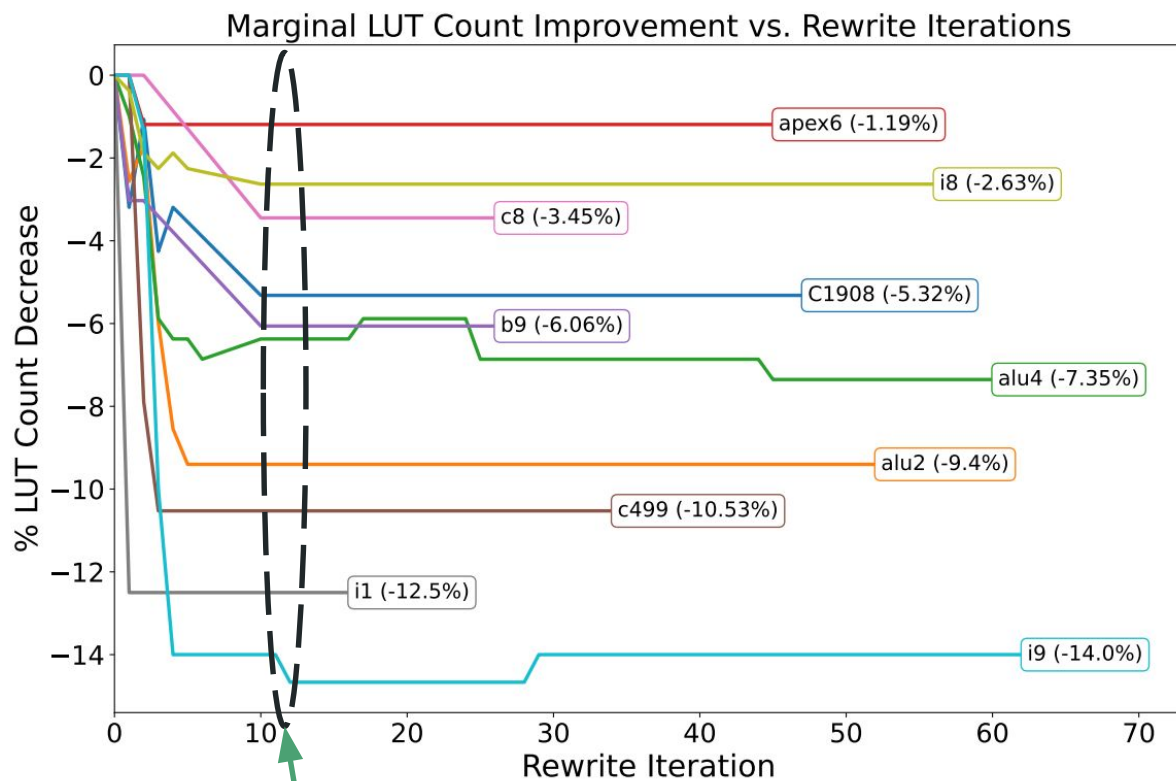
Figure 2: Number of gates in a 48-hour run of our tool applied to the preprocessed IWLS ex07 instance.

1. Reichl, F.-X., Slivovsky, F., & Szeider, S. (2023). Circuit Minimization with QBF-Based Exact Synthesis. Proceedings of the AAAI Conference on Artificial Intelligence

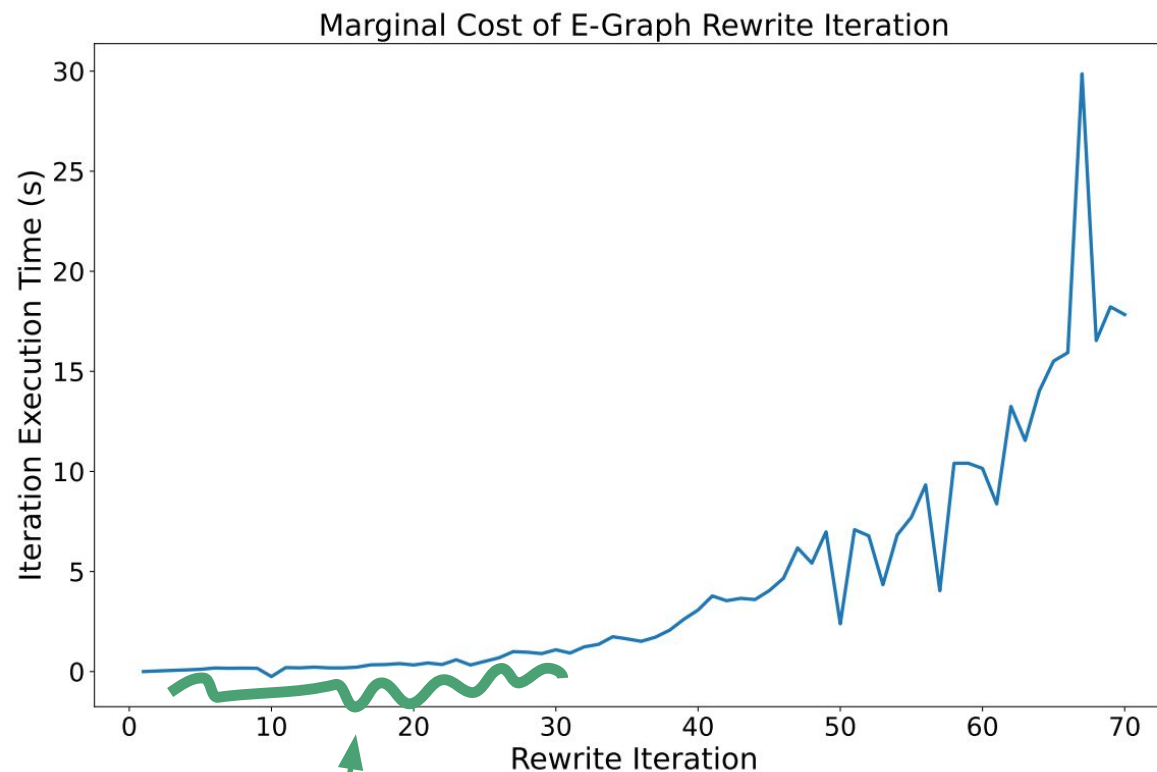
LUT Rewriting System



Results (cont.)



Empirically, 12 iterations is good.



The first 30 iterations are quick